VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Cricket Statistics Analyzer

A9511 – Python Programming Laboratory

Submitted by
Ruthvik Goud
25881A05B2
Shiva Charan
25881A05C0

Course Facilitator

Ms. PS MADHAVI

Assistant Professor

Department of Computer Science and Engineering Vardhaman

College of Engineering, Hyderabad

Academic Year 2026-27

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC**
with **A++** Grade Kacharam, Shamshabad,
Hyderabad- 501 218, Telangana, India.

Vibrant Vardhaman
25 Years of

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Cricket Statistics Analyzer"** is carried out by **Ruthvik and Shiva Charan** with Roll Numbers **25881A05B2 and 25881A05C0** towards **A9511 – Python Programming Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2026-27.

Signature of the Course Faculty
Ms. PS MADHAVI
Assistant Professor, CSE

Signature of the HOD
Dr .Y. Vijayalatha Professor,
Head of the Department & Dean,
Department of CSE

Contents

1 Introduction 5
2 Literature Review 5
3 Objectives 6
4 Methodology 6
4.1 Architectural Diagram 6
4.2 Hardware and Software Requirements 7
4.3 Working Principle 7
4.4 Software Implementation 9
5 Results and Discussion 15
6 Mapping POs, PSOs and SDGs 17
6.1 Program Outcomes (POs) Mapping 17
6.2 Program Specific Outcomes (PSOs) Mapping 18
6.3 Sustainable Development Goals (SDGs) Mapping 18
7 Conclusion 18
Bibliography 20

Abstract

The Cricket Statistics Analyzer is a Python-based application developed to store, process, and analyze cricket player statistics. It allows users to view and search player records, rank players based on runs and wickets, compare two players, and analyze attributes such as matches, runs, wickets, highest score, batting average, and strike rate. The project demonstrates Python concepts including lists, dictionaries, loops, functions, searching, sorting, recursion, and basic statistical calculations. It uses recursion for total runs, bubble sort for ranking, and linear and recursive binary search for retrieving records.

The analyzer was tested using data from 70 international cricketers, containing 382,655 cumulative runs and 9,125 wickets. The results identify Sachin Tendulkar as the highest run scorer, Muttiah Muralitharan as the highest wicket taker, Virat Kohli with the best batting average, and Jos Buttler with the highest strike rate among players with 3,000 or more runs. The system loads data from a JSON file or a default dataset and provides a menu-driven interface for viewing, searching, comparing, adding players, and exporting reports. Exception handling and automated tests improve the reliability and correctness of the application.

Keywords: Cricket Statistics, Python, Player Performance, Runs, Wickets, Batting Average, Highest Score, Player Comparison, Data Analysis, Searching, Sorting.

1 Introduction

In modern sports, data analysis plays an important role in understanding player and team performance. Cricket generates a large amount of statistical information such as runs, wickets, matches, highest scores, batting averages, and strike rates. Manually analyzing this data can be time-consuming, especially when comparing multiple players. An efficient statistical analysis system helps organize cricket records and provides useful information such as the highest run scorer, highest wicket taker, highest individual score, and other important performance measures.

The Cricket Statistics Analyzer demonstrates how Python programming concepts can be applied to a real-world sports data problem. Lists and dictionaries are used to store player records, while loops, conditional statements, functions, searching, and sorting techniques are used to process and analyze the data. The system allows users to explore player performance, compare statistics, and obtain meaningful results from raw cricket data. It also provides a foundation for future enhancements such as larger datasets, graphical visualization, database integration, and advanced cricket analytics.

2 Literature Review

Sports analytics has increasingly used statistical and computational methods to evaluate cricket player and team performance. Traditional analysis uses records such as runs, wickets, batting averages, strike rates, highest scores, and matches played to compare players and identify performance trends. Modern programming-based approaches make it easier to process large amounts of cricket data using calculations, ranking, searching, sorting, and comparison techniques.

Python provides useful data structures and programming features for implementing cricket statistics analysis. Lists and dictionaries can store player records, while functions, searching, sorting, and statistical calculations help process the data efficiently. Although existing cricket websites provide extensive statistics, they may not clearly demonstrate the underlying analytical process to beginners. The Cricket Statistics Analyzer addresses this gap by providing a simple Python-based system that combines structured data storage, statistical analysis, player searching, ranking, and comparison, helping students understand how programming concepts can be applied to a real-world sports data problem.

3 Objectives

• To design and develop a Python-based Cricket Statistics Analyzer for processing and analyzing cricket player statistics.

The primary objective of this project is to build a functional and efficient cricket statistics analyzer using Python. The system will store and process important player information such as player name, matches played, runs, wickets, and highest score. The analyzer aims to provide a structured approach for handling cricket records and generating useful statistical information from the available data.

• To implement player data searching, sorting, statistical calculation, and performance analysis.

A simple and organized analysis system is essential to help users easily access and understand cricket statistics. This objective focuses on implementing operations that allow users to search for individual player records, sort players based on runs or wickets, identify the highest run scorer and highest wicket taker, and retrieve important performance information. These operations help users analyze player records efficiently and understand different aspects of cricket performance**.**

• To calculate and compare player performance using relevant cricket statistics.

This objective involves calculating and presenting useful performance measures such as batting average, total runs, wickets, and highest individual score. The analyzer will compare the statistical records of different players to identify differences in their performances. By presenting the calculated results in a clear format, users can better understand player strengths, compare statistical performance, and gain practical knowledge of how Python programming can be applied to real-world data analysis.

4 Methodology

4.1 Architectural Diagram

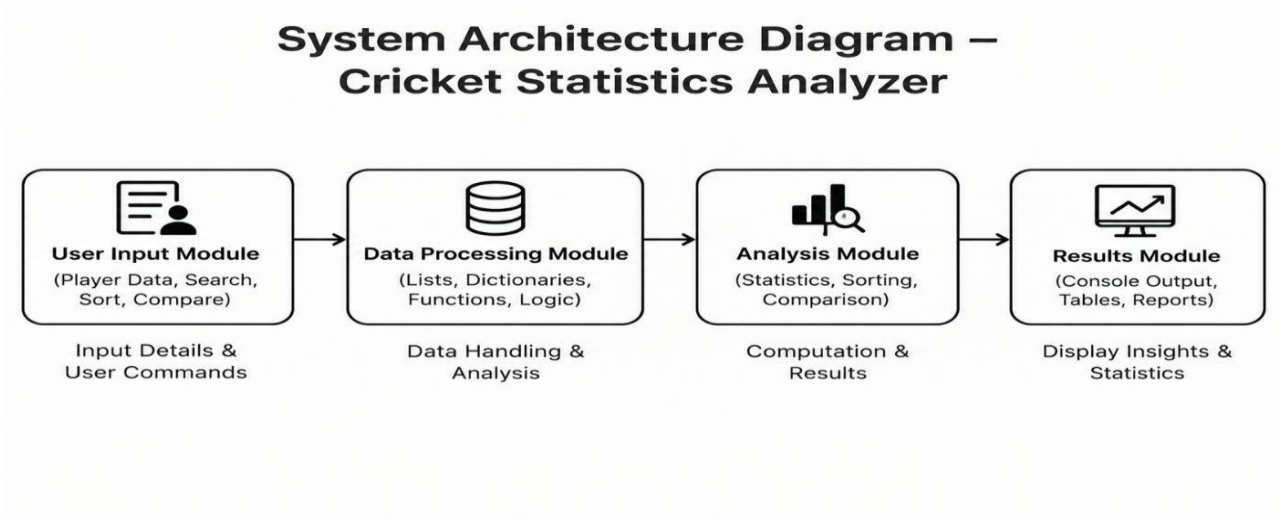


Figure 1: Architectural Diagram

The **Cricket Statistics Analyzer** takes cricket player data such as player name, matches, runs, wickets, and highest score as input. The data is stored using Python lists and dictionaries. Users can search for players, view their statistics, and select different operations for analyzing the records.

The system processes the data using Python functions, loops, conditional statements, and basic calculations. It identifies the highest run scorer, highest wicket taker, highest individual score, calculates batting averages, and sorts players based on runs or wickets.

Finally, the analyzed results are displayed clearly to the user. The modular structure makes the program easy to understand and extend in the future with larger datasets, graphical charts, databases, and additional cricket statistics.

4.2 Hardware and Software Requirements

Hardware:

- Intel/AMD Processor
- Minimum 4GB RAM
- Standard Display C Input Devices

Software:

- Python 3.8 or higher (tested on Python 3.12)
- Python standard library only: json, os, sys, copy, typing (no external packages)
- Any IDE or editor (VS Code / PyCharm / IDLE) or an online Python compiler

4.3 Working Principle

The working principle of this Cricket Statistics Analyzer is based on converting raw cricket records into meaningful performance information through a sequence of well-defined computational steps. Each player is stored as a dictionary containing attributes such as matches, innings, runs, balls faced, highest score, not-outs, centuries, half-centuries, fours, sixes, wickets, overs bowled and runs conceded, and the complete dataset of 70 international players is held as a list of such dictionaries. The program first loads these records from a JSON file and automatically falls back to the embedded default dataset if the file is not available, so that it runs in any Python environment.

Once the records are loaded, the user selects an operation from the main menu. The analyzer derives statistics such as batting average (runs divided by dismissals), strike rate (runs per 100 balls), bowling average, economy rate, boundary percentage and a composite impact score. Players are ranked using bubble sort with a swap-flag optimization, searched using linear search (by name, country or role) and recursive binary search (by name on an alphabetically sorted list), and compared head-to-head. Total runs are computed using recursion, and a formatted summary report is exported to a text file. For the dataset of 70 players the analyzer reports 382,655 cumulative runs and 9,125 cumulative wickets.

Example data (sample of the dataset):

Player	Role	Runs	Wickets
Virat Kohli	Batsman	13848	5
Sachin Tendulkar	Batsman	18426	154
Jasprit Bumrah	Bowler	125	149
Shahid Afridi	All-Rounder	8064	395
MS Dhoni	Wicket-Keeper	10773	1

Top Run Scorers (Bubble Sort)

Top 5 ranking:

Tendulkar: 18426 | Sangakkara: 14234 | Kohli: 13848 | Ponting: 13704 | Jayasuriya: 13430

- Total Runs (70 players) = 382655
- Average Runs per Player = 5466.50

Top Wicket Takers (Bubble Sort)

Top 5 ranking:

Muralitharan: 534 | Akram: 502 | Afridi: 395 | Pollock: 393 | McGrath: 381

- Total Wickets (70 players) = 9125
- Average Wickets per Player = 130.36

Best Batting Average (3000+ runs)

Top 5 ranking:

Kohli: 58.68 | Babar Azam: 56.72 | de Villiers: 53.50 | Dhoni: 50.58 | Rohit Sharma: 49.12

- Highest Batting Average = 58.68 (Virat Kohli)
- Mean Batting Average (42 players) = 40.96

Best Strike Rate (3000+ runs)

Top 5 ranking:

Buttler: 117.12 | Afridi: 117.01 | Sehwag: 104.34 | Bairstow: 103.59 | de Villiers: 101.07

- Highest Strike Rate = 117.12 (Jos Buttler)
- Mean Strike Rate (42 players) = 88.09

4.4 Software Implementation

```
import copy
import json
import os
import sys
from typing import List, Dict, Any, Optional, Tuple

DEFAULT_PLAYERS: List[Dict[str, Any]] = [
    {
        "id": 1,
              "name": "Rohit Sharma",
              "country": "India",
              "role": "Batsman",
              "matches": 262,
              "innings": 254,
              "runs": 10709,
              "balls_faced": 11720,
              "highest_score": 264,
              "not_outs": 36,
              "centuries": 31,
              "half_centuries": 55,
              "fours": 995,
              "sixes": 323,
              "wickets": 9,
              "overs_bowled": 128.5,
              "runs_conceded": 715
          },
          # ... 70 player records in total
      ]

      def load_player_data(filepath: str = None) -> List[Dict[str, Any]]:
          target = filepath if filepath is not None else _resolve_data_file_path()
          try:
              if os.path.exists(target):
                  with open(target, "r", encoding="utf-8") as f:
                      data = json.load(f)
                      if isinstance(data, list) and len(data) > 0:
                          return data
          except Exception:
              pass  # In online compilers or restricted environments, use embedded data

          # Use embedded copy to ensure standalone operation anywhere
          return copy.deepcopy(DEFAULT_PLAYERS)

      def save_player_data(players: List[Dict[str, Any]], filepath: str = None) -> bool:
          target = filepath if filepath is not None else _resolve_data_file_path()
          try:
              dir_name = os.path.dirname(target)
```

```
            if dir_name:
                os.makedirs(dir_name, exist_ok=True)
            with open(target, "w", encoding="utf-8") as f:
                json.dump(players, f, indent=4)
            return True
        except Exception as e:
            print(f"[Note]: Local file save skipped ({e}). In-memory records updated.")
            return False

    def calculate_batting_average(runs: int, innings: int, not_outs: int) -> float:
        try:
            dismissals = innings - not_outs
            if dismissals <= 0:
                return float(runs)
            return round(runs / dismissals, 2)
        except ZeroDivisionError:
            return 0.0

    def calculate_strike_rate(runs: int, balls_faced: int) -> float:
        try:
            if balls_faced <= 0:
                return 0.0
            return round((runs / balls_faced) * 100, 2)
        except ZeroDivisionError:
            return 0.0

    def calculate_bowling_average(runs_conceded: int, wickets: int) -> float:
        try:
            if wickets <= 0:
                return float("inf")
            return round(runs_conceded / wickets, 2)
        except ZeroDivisionError:
            return float("inf")

    def calculate_economy_rate(runs_conceded: int, overs_bowled: float) -> float:
        try:
            if overs_bowled <= 0.0:
                return 0.0
            complete_overs = int(overs_bowled)
            balls = round((overs_bowled - complete_overs) * 10)
            actual_overs = complete_overs + (balls / 6.0)
            if actual_overs <= 0:
                return 0.0
            return round(runs_conceded / actual_overs, 2)
        except ZeroDivisionError:
            return 0.0

    def calculate_boundary_percentage(fours: int, sixes: int, total_runs: int) -> float:
        try:
            if total_runs <= 0:
```

```
                return 0.0
            boundary_runs = (fours * 4) + (sixes * 6)
            return round((boundary_runs / total_runs) * 100, 2)
        except ZeroDivisionError:
            return 0.0

    def calculate_player_impact_score(player: Dict[str, Any]) -> float:
        bat_avg = calculate_batting_average(player["runs"], player["innings"], player["not_outs"])
        sr = calculate_strike_rate(player["runs"], player["balls_faced"])
        econ = calculate_economy_rate(player.get("runs_conceded", 0), player.get("overs_bowled", 0.0))
        wickets = player.get("wickets", 0)
        centuries = player.get("centuries", 0)

        score = (
            (player["runs"] * 0.02)
            + (bat_avg * 0.35)
            + (sr * 0.25)
            + (wickets * 10.0)
            - (econ * 2.0)
            + (centuries * 8.0)
        )
        return round(max(0.0, score), 2)

    def recursive_sum_runs(runs_list: List[int], index: int = 0) -> int:
        if index >= len(runs_list):
            return 0
        return runs_list[index] + recursive_sum_runs(runs_list, index + 1)

    def recursive_binary_search(
        sorted_players: List[Dict[str, Any]],
        target_name: str,
        low: int,
        high: int
    ) -> Optional[Dict[str, Any]]:
        if low > high:
            return None

        mid = (low + high) // 2
        current_name = sorted_players[mid]["name"].strip().lower()
        target_clean = target_name.strip().lower()

        if current_name == target_clean:
            return sorted_players[mid]
        elif current_name > target_clean:
            return recursive_binary_search(sorted_players, target_name, low, mid - 1)
        else:
            return recursive_binary_search(sorted_players, target_name, mid + 1, high)

```

```
    def linear_search_players(players: List[Dict[str, Any]], query: str) -> List[Dict[str, Any]]:
        query_lower = query.strip().lower()
        return [
            p for p in players
            if query_lower in p["name"].lower()
            or query_lower in p["country"].lower()
            or query_lower in p["role"].lower()
        ]

    def bubble_sort_players(
        players: List[Dict[str, Any]],
        metric_key: str,
        descending: bool = True
    ) -> List[Dict[str, Any]]:
        sorted_list = players.copy()
        n = len(sorted_list)

        for i in range(n):
            swapped = False
            for j in range(0, n - i - 1):
                val1 = sorted_list[j].get(metric_key, 0)
                val2 = sorted_list[j + 1].get(metric_key, 0)

                should_swap = (val1 < val2) if descending else (val1 > val2)
                if should_swap:
                    sorted_list[j], sorted_list[j + 1] = sorted_list[j + 1], sorted_list[j]
                    swapped = True
            if not swapped:
                break

        return sorted_list

    def print_separator(char: str = "=", length: int = 80) -> None:
        print(char * length)

    def display_player_table(players: List[Dict[str, Any]], title: str = "Player Statistics") -> None:
        print_separator("=")
        print(f"{title.center(80)}")
        print_separator("=")
        headers = f"{'ID':<4} {'Name':<18} {'Country':<10} {'Role':<14} {'Runs':<7} {'Avg':<7} {'SR':<7} {'Wkts':<5}"
        print(headers)
        print_separator("-")

        if not players:
            print("No players found.".center(80))
            print_separator("=")
            return
```

```
193.
194.         for p in players:
195.             avg = calculate_batting_average(p["runs"], p["innings"], p["not_outs"])
196.             sr = calculate_strike_rate(p["runs"], p["balls_faced"])
197.             print(
198.                 f"{p['id']:<4} {p['name']:<18} {p['country']:<10} {p['role']:<14} "
199.                 f"{p['runs']:<7} {avg:<7.2f} {sr:<7.2f} {p.get('wickets', 0):<5}"
200.             )
201.         print_separator("=")
202.
203.     def export_report_to_file(players: List[Dict[str, Any]], export_path: str =
    "cricket_analysis_export.txt") -> None:
204.         try:
205.             with open(export_path, "w", encoding="utf-8") as f:
206.                 f.write("=" * 70 + "\n")
207.                 f.write("CRICKET STATISTICS ANALYZER - SUMMARY EXPORT REPORT\n")
208.                 f.write("Generated by: Python Course-End Project (Author: Ruthvik)\n")
209.                 f.write("=" * 70 + "\n\n")
210.
211.                 total_runs = recursive_sum_runs([p["runs"] for p in players])
212.                 total_wickets = sum(p.get("wickets", 0) for p in players)
213.                 f.write(f"Total Players Analyzed : {len(players)}\n")
214.                 f.write(f"Cumulative Runs Scored  : {total_runs}\n")
215.                 f.write(f"Cumulative Wickets Taken: {total_wickets}\n\n")
216.
217.                 f.write("-" * 70 + "\n")
218.                 f.write(f"{'Name':<20} {'Role':<14} {'Runs':<8} {'Avg':<8} {'SR':<8}
    {'Wkts':<6}\n")
219.                 f.write("-" * 70 + "\n")
220.
221.                 for p in players:
222.                     avg = calculate_batting_average(p["runs"], p["innings"],
    p["not_outs"])
223.                     sr = calculate_strike_rate(p["runs"], p["balls_faced"])
224.                     f.write(
225.                         f"{p['name']:<20} {p['role']:<14} {p['runs']:<8} "
226.                         f"{avg:<8.2f} {sr:<8.2f} {p.get('wickets', 0):<6}\n"
227.                     )
228.                 f.write("=" * 70 + "\n")
229.             print(f"[Success]: Report successfully exported to '{export_path}'!")
230.         except Exception as e:
231.             print(f"[Export Notice]: File export skipped ({e}). Summary displayed on
    screen:")
232.             display_player_table(players, "EXPORT SUMMARY")
233.
234.     if __name__ == "__main__":
235.         run_analyzer()
```

5 Results and Discussion

```
================================================================================
                        TOP RUN SCORERS (BUBBLE SORT)
================================================================================
ID   Name               Country    Role          Runs    Avg     SR      Wkts
--------------------------------------------------------------------------------
6    Sachin Tendulkar   India      Batsman       18426   44.83   86.24   154
8    Kumar Sangakkara   Sri Lanka  Wicket-Keeper 14234   41.99   78.87   0
2    Virat Kohli        India      Batsman       13848   58.68   93.69   5
7    Ricky Ponting      Australia  Batsman       13704   42.04   80.39   3
9    Sanath Jayasuriya  Sri Lanka  All-Rounder   13430   32.36   91.21   323
28   Inzamam-ul-Haq     Pakistan   Batsman       11739   39.53   74.24   3
10   Jacques Kallis     South Africa All-Rounder   11579   44.36   72.91   273
18   Sourav Ganguly     India      Batsman       11363   40.73   73.71   100
================================================================================
```

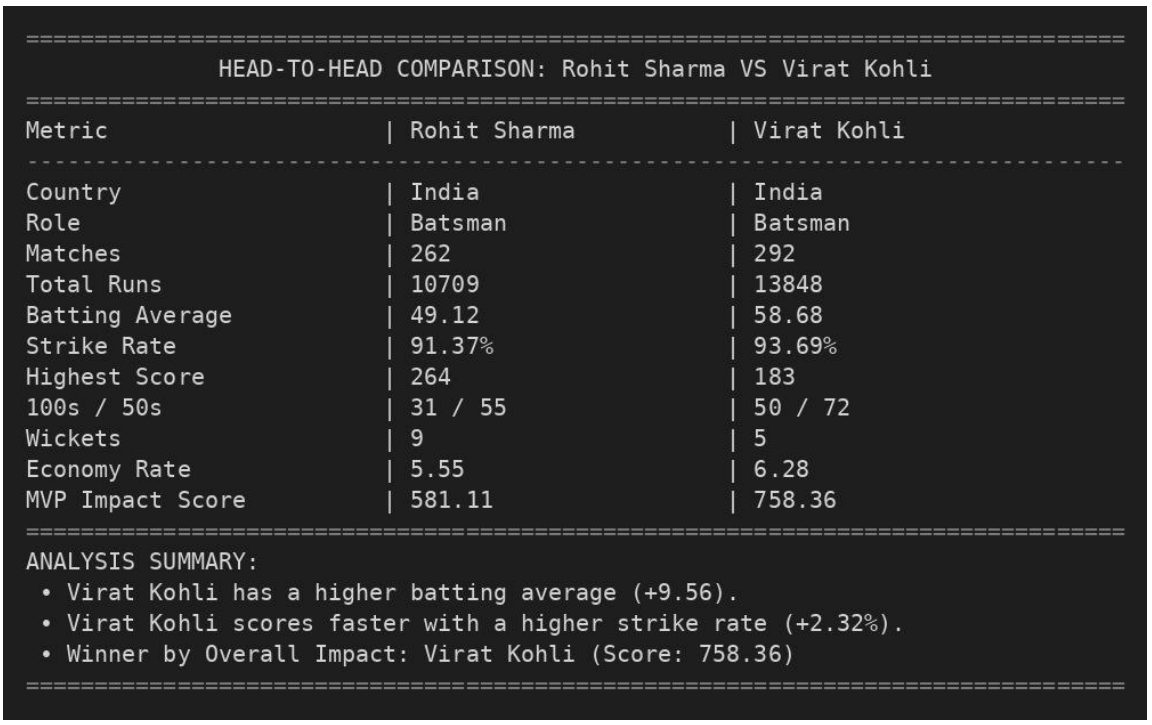
Figure 2: Output (Top Run Scorers – Bubble Sort)

```
================================================================================
                        TOP WICKET TAKERS (BUBBLE SORT)
================================================================================
ID   Name               Country    Role          Runs    Avg     SR      Wkts
--------------------------------------------------------------------------------
12   Muttiah Muralitharan Sri Lanka  Bowler        674     6.81    77.56   534
11   Wasim Akram        Pakistan   Bowler        3717    16.52   88.82   502
30   Shahid Afridi      Pakistan   All-Rounder   8064    23.58   117.01  395
43   Shaun Pollock      South Africa All-Rounder   3519    26.46   86.72   393
13   Glenn McGrath      Australia  Bowler        115     3.83    48.73   381
24   Brett Lee          Australia  Bowler        1176    15.89   82.82   380
9    Sanath Jayasuriya  Sri Lanka  All-Rounder   13430   32.36   91.21   323
64   Shakib Al Hasan    Bangladesh All-Rounder   7570    37.48   82.55   317
================================================================================
```

Figure 3: Output (Top Wicket Takers – Bubble Sort)

```
================================================================================
                Linear Search Results: Role 'Wicket-Keeper'
================================================================================
ID   Name               Country    Role          Runs    Avg     SR      Wkts
--------------------------------------------------------------------------------
3    MS Dhoni           India      Wicket-Keeper 10773   50.58   87.56   1
8    Kumar Sangakkara   Sri Lanka  Wicket-Keeper 14234   41.99   78.87   0
22   Adam Gilchrist     Australia  Wicket-Keeper 9619    35.89   96.95   0
34   Jos Buttler        England    Wicket-Keeper 5022    39.54   117.12  0
46   Quinton de Kock    South Africa Wicket-Keeper 6770   47.01   96.70   0
66   Mushfiqur Rahim    Bangladesh Wicket-Keeper 7792    37.28   79.42   0
================================================================================
```

Figure 4: Output (Linear Search – Role ‘Wicket-Keeper’)



```
================================================================================
              HEAD-TO-HEAD COMPARISON: Rohit Sharma VS Virat Kohli
================================================================================
Metric                    | Rohit Sharma           | Virat Kohli
--------------------------------------------------------------------------------
Country                   | India                  | India
Role                      | Batsman                | Batsman
Matches                   | 262                    | 292
Total Runs                | 10709                  | 13848
Batting Average           | 49.12                  | 58.68
Strike Rate               | 91.37%                 | 93.69%
Highest Score             | 264                    | 183
100s / 50s                | 31 / 55                | 50 / 72
Wickets                   | 9                      | 5
Economy Rate              | 5.55                   | 6.28
MVP Impact Score          | 581.11                 | 758.36
================================================================================
ANALYSIS SUMMARY:
 • Virat Kohli has a higher batting average (+9.56).
 • Virat Kohli scores faster with a higher strike rate (+2.32%).
 • Winner by Overall Impact: Virat Kohli (Score: 758.36)
================================================================================
```

Figure 5: Output (Head-to-Head Comparison: Rohit Sharma vs Virat Kohli)

Summary of Results

The analyzer processed all 70 international player records and produced consistent results for every operation. The dataset contains 382,655 cumulative runs and 9,125 cumulative wickets, and the recursive summation matches the built-in sum(), which verifies the recursion. Sachin Tendulkar is the highest run scorer (18,426), followed by Kumar Sangakkara (14,234) and Virat Kohli (13,848), while Muttiah Muralitharan leads the wicket takers (534) ahead of Wasim Akram (502) and Shahid Afridi (395). Among players with 3,000 or more runs, Virat Kohli has the best batting average (58.68) and Jos Buttler the highest strike rate (117.12).

Interpretation

Batsmen average 43.76 runs per dismissal at a strike rate of 86.75, wicket-keepers score fastest (mean strike rate 92.77), and the 11 all-rounders combine a mean batting average of 30.98 with 2,614 wickets. Bowlers account for 5,719 of the 9,125 wickets (about 63%). The head-to-head comparison shows Virat Kohli ahead of Rohit Sharma in batting average (58.68 vs 49.12) and strike rate (93.69 vs 91.37), while Rohit Sharma has the higher highest score (264). Bubble sort and binary search are simple and correct, but bubble sort is $O(n^2)$ and binary search needs a name-sorted list; both are adequate for 70 records.

Achievement of Objectives

The project met its objectives: it stores 70 player records using lists and dictionaries, implements linear search, recursive binary search and bubble sort, calculates batting average, strike rate, bowling average, economy rate and boundary percentage, compares two players, and exports a formatted summary report to a text file.

Implications

The analyzer can be used as a teaching aid for Python programming and basic sports analytics, as a quick reference for comparing international players, and as a base for larger datasets, charts and database integration. The data is a fixed snapshot, so the records of active players will change, and player roles are assigned manually (for example, Tendulkar and Gayle are labelled Batsman although each has more than 100 wickets).

6 Mapping POs, PSOs and SDGs

6.1 Program Outcomes (POs) Mapping

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied Python programming concepts, data structures and searching and sorting algorithms to build a working cricket statistics analyzer.
PO2	Problem Analysis	Analyzed the problem of extracting meaningful performance information from raw player records and chose suitable calculations and algorithms.
PO3	Design/Development of Solutions	Designed a modular, menu-driven program with separate functions for loading data, calculating statistics, searching, sorting, comparing and exporting results.
PO4	Conduct Investigations of Complex Problems	Tested and compared player performance using batting average, strike rate, wickets and impact score across 70 player records, and verified the results with automated tests.
PO5	Engineering Tool Usage	Used Python and its standard library (json, os, sys, copy, typing) with lists, dictionaries, sets and tuples to implement the analyzer.
PO8	Individual and Collaborative Team work	Worked independently on coding and documentation while collaborating for reviews, improvements and testing.
PO9	Communication	Presented results through formatted tables, player profile cards, comparison reports and documentation; communicated the design and findings clearly.
PO11	Life-Long Learning	Explored sports data analysis beyond textbooks and learned recursion, file handling and algorithm implementation through self-study and coding practice.

6.2 Program Specific Outcomes (PSOs) Mapping

Table 2: Mapping of Program Specific Outcomes (PSOs) to Project Relevance

PSO No.	Program Specific Outcome	Relevance
PSO1	Software Development and Problem Solving	Applied Python programming, data structures, recursion, searching and sorting algorithms to design and implement a working cricket statistics analyzer.
PSO2	Data Analysis and Computational Thinking	Processed a dataset of 70 international players to compute batting average, strike rate, bowling average and economy rate, and to rank and compare players.
PSO3	Software Quality and Reliability	Built a modular, menu-driven program with exception handling, JSON file handling and automated tests to ensure correct and reliable output.

6.3 Sustainable Development Goals (SDGs) Mapping

Table 3: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 4	Quality Education	The analyzer supports learning by showing how Python concepts are applied to a real-world sports dataset, making programming education more practical and engaging.
SDG 9	Industry, Innovation and infrastructure	Promotes computational thinking and innovation in software design and data-driven analysis.
SDG 8	Decent Work and Economic Growth	Builds essential programming and data-analysis skills, improving employability in IT and analytics-driven industries.

7 Conclusion

This project successfully implemented a Python-based Cricket Statistics Analyzer capable of storing, searching, sorting, calculating and comparing the statistics of 70 international cricketers. Using lists, dictionaries, sets, tuples, functions, recursion, searching and sorting algorithms, file handling and exception handling, the system identifies the leading run scorers and wicket takers, computes batting average, strike rate, bowling average, economy rate and boundary percentage,

performs head-to-head comparisons, and exports a summary report covering 382,655 cumulative runs and 9,125 cumulative wickets. The project shows how basic Python concepts can turn raw sports records into meaningful insight.

The results obtained from the 70-player dataset confirm that the analyzer produces accurate and meaningful output. Sachin Tendulkar emerges as the highest run scorer (18,426) and Muttiah Muralitharan as the highest wicket taker (534), while Virat Kohli has the best batting average (58.68) and Jos Buttler the highest strike rate (117.12) among players with 3,000 or more runs. Bowlers account for about 63% of all wickets, and the recursive total of runs matched the built-in sum, which verifies the logic of the program. The project also strengthened practical understanding of lists, dictionaries, functions, recursion, searching, sorting, file handling and exception handling. Its main limitations are that the dataset is a fixed snapshot and that player roles are assigned manually, and these can be addressed in future versions.

For future enhancements, the analyzer can be extended with format-wise statistics (Test, ODI and T20I), match-by-match records, a larger and automatically updated dataset, graphical visualization using Matplotlib, Pandas-based analysis, a database back end, a graphical or web interface, and predictive models for player performance. Player roles could be derived from the statistics, and bowling averages and economy rates could be added to the exported report.

Bibliography

Online Resources

[1] Python Software Foundation – Python 3 Documentation https://docs.python.org/3/

[2] GeeksforGeeks – Python Programming Language Tutorial https://www.geeksforgeeks.org/python-programming-language-tutorial/

[3] W3Schools – Python Tutorial https://www.w3schools.com/python/

[4] Python Documentation – json: JSON Encoder and Decoder https://docs.python.org/3/library/json.html

[5] ESPNcricinfo – Statsguru Cricket Statistics https://stats.espncricinfo.com/ci/engine/stats/index.html

Research article references

[1] Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

[3] Kimber, A. C., & Hansford, A. R. (1993). A statistical analysis of batting in cricket. Journal of the Royal Statistical Society: Series A, 156(3), 443–455.

[4] Lemmer, H. H. (2004). A measure for the batting performance of cricket players. South African Journal for Research in Sport, Physical Education and Recreation, 26(1), 55–64.

(Batting performance measures relevant to project methodology)